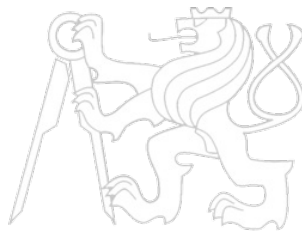


# Middleware Architectures 2

## Lecture 3: Security

**doc. Ing. Tomáš Vitvar, Ph.D.**

tomas@vitvar.com • @TomasVitvar • <https://vitvar.com>



Czech Technical University in Prague

Faculty of Information Technologies • Software and Web Engineering • <https://vitvar.com/lectures>



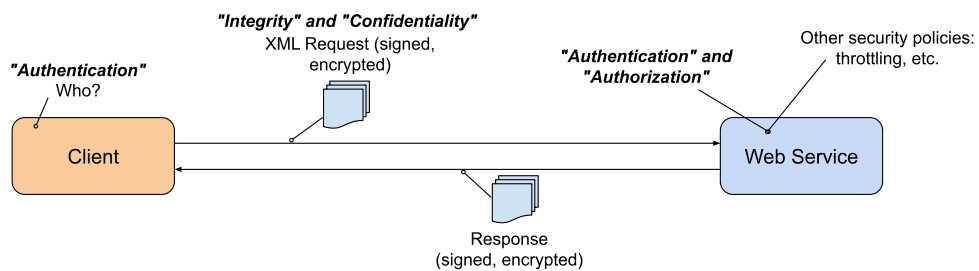
Modified: Mon Mar 16 2026, 14:39:07  
Humla v1.0

## Overview

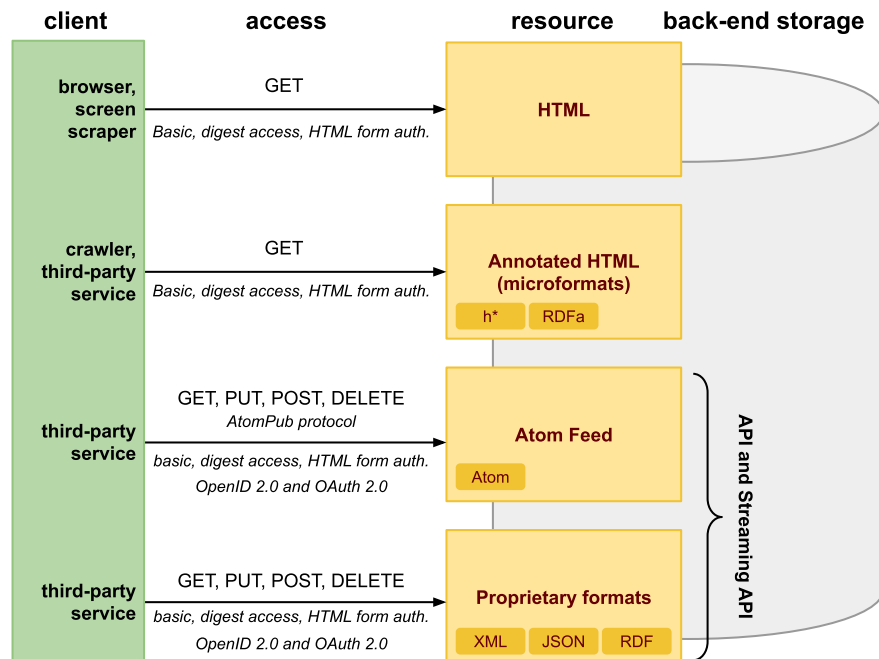
- **Security Concepts**
  - *Access and Refresh Tokens*
  - *Authorization Models*
- Transport Level Security
- JSON Web Token
- OAuth 2.0
- OpenID

## Web Service Security Concepts

- Securing the client-server communication
  - *Message-level security*
  - *Transport-level security*
- Ensure
  - *Authentication* – *verify a client's identity*
  - *Authorization* – *rights to access resources*
  - *Message Confidentiality* – *keep message content secret*
  - *Message Integrity* – *message content does not change during transmission*
  - *Non-repudiation* – *proof of integrity and origin of data*



## Data on the Web



## Authentication and Authorization

- Authentication
  - *verification of user's identity*
- Authorization
  - *verification that a user has rights to access a resource*
- Standard: HTTP authentication
  - *HTTP defines several main options*
    - *Basic Access Authentication*
    - *Digest Access Authentication*
    - *Bearer tokens to access OAuth 2.0-protected resources*
    - *Mutual authentication using password-based when server knows the user's encrypted password*
  - *Basic and Digest are defined in*
    - *RFC 2616: Hypertext Transfer Protocol – HTTP/1.1*
    - *RFC 2617: HTTP Authentication: Basic and Digest Access Authentication*
- Custom/proprietary: use of cookies

## Overview

- Security Concepts
  - *Access and Refresh Tokens*
  - *Authorization Models*
- Transport Level Security
- JSON Web Token
- OAuth 2.0
- OpenID

## Access Tokens vs Refresh Tokens

- **Access token**
  - Credential presented to an API/resource server to authorize requests
  - **Short-lived** (minutes to ~1 hour): limits damage if leaked
  - Sent frequently
    - In an HTTP header (e.g., **Authorization: Bearer**)
    - In a cookie or a request body
  - Opaque (generated by the server) or self-contained (e.g., JWT)
- **Refresh token**
  - Longer-lived credential used to obtain new access tokens without re-login
  - Sent only to the token service / authorization server (not to every API)
  - A **high-value secret**
    - Closer to a password than an access token

## Token Lifetimes: Security vs Usability

- Why access tokens are short-lived
  - If an access token leaks, attacker's window is limited by expiry
  - Reduces dependency on revocation mechanisms
- Why refresh tokens exist
  - Users shouldn't re-authenticate often
  - Refresh token enables "silent re-auth" / session continuation
- Common design pattern
  - Access token: short TTL, used on every API call
  - Refresh token: long TTL, used rarely to mint new access tokens

## Rotation

- **Access token rotation**
  - Usually “rotates” naturally by minting a new access token when the old one expires
  - Old access token may remain valid until expiry (unless you have server-side introspection/revocation)
- **Refresh token rotation (one-time use refresh tokens)**
  - Client uses refresh token  $R1$
  - Token service returns  $A2 + R2$  and invalidates  $R1$
- **Why rotate refresh tokens**
  - Limits usefulness of a stolen refresh token
  - Enables **reuse detection**
    - using an old refresh token again signals theft/replay

## Refresh Token Reuse Detection

- **Threat model**
  - Attacker steals refresh token  $R1$ 
    - e.g. malware, XSS, insecure storage, logs
  - Both attacker and real client may try to refresh
- **Rotation + reuse detection**
  - Legitimate client refreshes:  $R1$ 
    - gets  $R2$  and server invalidates  $R1$
  - If attacker later submits  $R1$  again
    - server detects reuse of an invalidated token
    - the whole session can be revoked, force re-authentication, sends alert

## Binding Tokens to Context

- IP binding (soft or strict)
  - Accept token only if request IP matches expected IP / range
  - Tradeoff: reduces replay but can break real users
    - mobile IP changes, VPNs
  - Often used as **soft binding**
    - accept token but trigger extra checks
- Device binding / proof-of-possession
  - Tie token to a device-held key
  - Stronger than IP binding but more complex to deploy
- Client binding
  - Refresh tokens usable only by the intended client app
    - Client identity / audience checks

## General Protection Mechanisms

- Storage & handling
  - Protect refresh tokens like passwords
  - Never log tokens (server logs, analytics, crash reports)
  - Send refresh tokens only to the token service
- Transport security
  - HTTPS/TLS everywhere; secure cookie flags
  - Consider HSTS
- Server-side controls
  - Revocation, rate-limit refresh attempts
  - Least privilege scopes/permissions
    - audience restrictions (token for API A  $\neq$  API B)

## Overview

- Security Concepts
  - *Access and Refresh Tokens*
  - *Authorization Models*
- Transport Level Security
- JSON Web Token
- OAuth 2.0
- OpenID

## RBAC, ABAC, ACL

- Overall concepts
  - *Triple: subject (user) -- operation -- object (resource)*
- **ACL** (Access Control List)
  - *Each resource stores a list of **who can do what***
    - *Alice=owner; Bob=read; TeamX=write*
  - *Typical in file systems and document sharing/collaboration tools*
- **RBAC** (Role-Based Access Control)
  - *Users are assigned **roles***
    - *e.g. admin, editor, viewer*
    - *roles grant permissions*
  - *Common in enterprise apps and SaaS admin/tenant management*
- **ABAC** (Attribute-Based Access Control)
  - *Decision based on **attributes** of user/resource/context (dept, clearance, time, device, region)*
  - *Large policy-driven organizations*

## Role-Based Access Control (RBAC)

- Goal
  - Control *who can do what* by assigning *roles* to users
  - Roles map to a set of *permissions* (operations on resources)
- Core concepts
  - *User*: Alice
  - *Role*: admin, editor, viewer
  - *Permission*: invoice:read, invoice:approve, user:invite
  - *Assignment*: Alice has role editor  $\Rightarrow$  Alice inherits editor permissions
- Why RBAC is used
  - Scales better than per-user ACLs
  - Easier audits: “Who are all the admins?”
  - Supports least privilege via limited roles

## RBAC Example: Roles $\rightarrow$ Permissions

- Define roles and permissions mappings

```
1 viewer:
2   project:read
3   issue:read
4 editor:
5   project:read
6   issue:read
7   issue:create
8   issue:update
9 admin:
10  project:read
11  project:delete
12  issue:*
13  user:invite
14  user:remove
```

- Assign roles to users

```
1 Alice -> editor
2 Bob -> viewer
3 Peter -> (no role)
```



## RBAC in an HTTP API (Enforcement)

- Permissions can be checked at the server for each request
  - Step 1: authenticate user (session cookie / OAuth / JWT)
  - Step 2: load roles/permissions
  - Step 3: authorize the action (allow/deny)
- Example API endpoints
  - 1 GET /projects/123 → requires project:read
  - 2 POST /projects/123/issues → requires issue:create
  - 3 PUT /projects/123/issues/9 → requires issue:update
  - 4 POST /org/users/invite → requires user:invite
- Example authorization decision
  - Alice (editor) can POST /projects/123/issues
  - Bob (viewer) can only read issues, not create/update
  - Admins can invite/remove users

## RBAC Shortcomings and Best Practices

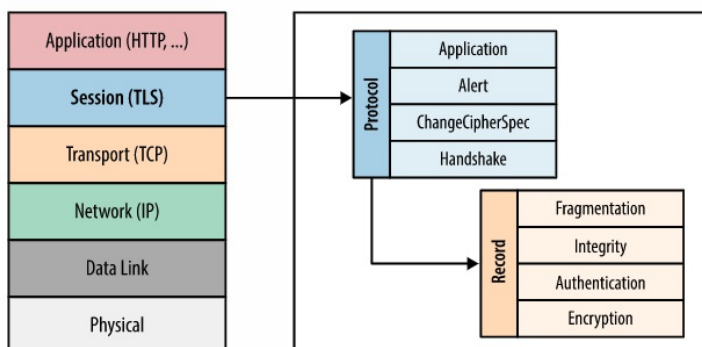
- Shortcomings
  - *Confusing roles with resources*: global admin vs project-admin
  - Having permission issue:read is not enough
    - must also check access to issue's project
- Best practices
  - Deny by default; keep roles small and explicit
  - Use scopes (RBAC + scope): role=editor in project 123
  - Log authorization decisions for audits
  - Tests
    - viewer have read-only access
    - editor cannot invite users

## Overview

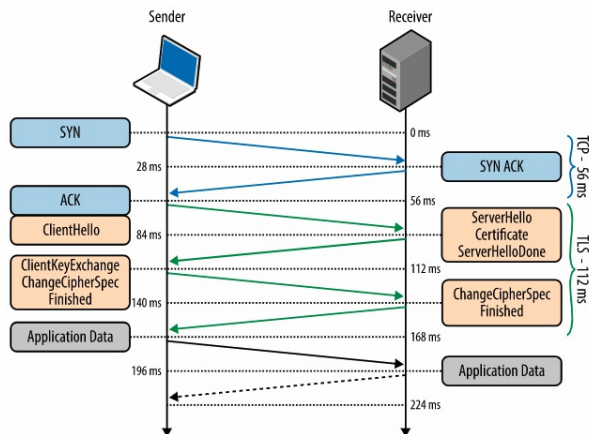
- Security Concepts
- **Transport Level Security**
- JSON Web Token
- OAuth 2.0
- OpenID

## Overview

- SSL and TLS
  - *SSL and TLS is used interchangeably*
  - *SSL 3.0 developed by Netscape*
  - *IETF standardization of SSL 3.0 is TLS 1.0*
    - *TLS 1.0 is upgrade of SSL 3.0*
  - *Due to security flaws in TLS 1.0, TLS 1.1 and TLS 1.2 were created*
- TLS layer



# TLS Handshake Protocol



- **TLS Handshake**

*56 ms: ClientHello, TLS protocol version, list of ciphersuites, TLS options*

*84 ms: ServerHello, TLS protocol version, ciphersuite, certificate*

*112 ms: RSA or Diffie-Hellman key exchange*

*140 ms: Message integrity checks, sends encrypted "Finished" message*

*168 ms: Decrypts the message, app data can be sent*

# TLS and Proxy Servers

- **TLS Offloading**

- Inbound TLS connection, plain outbound connection
- Proxy can inspect messages

- **TLS Bridging**

- Inbound TLS connection, new outbound TLS connection
- Proxy can inspect messages

- **End-to-End TLS (TLS pass-through)**

- TLS connection is passed-through the proxy
- Proxy cannot inspect messages

- **Load balancer**

- Can use TLS offloading or TLS bridging
- Can use TLS pass-through with help of Server Name Indication (SNI)

# HTTP Strict Transport Security

- Problem
  - Initial **http://** request (or insecure redirect) enables SSL-stripping
  - Mixed links/bookmarks can accidentally use HTTP even if site supports HTTPS
- Solution
  - HSTS: HTTP Strict Transport Security
  - Server tells browser: "Always use HTTPS for this site for N seconds"
  - Browser automatically upgrades future requests to HTTPS before sending traffic
- Header

```
1 | Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```
- Parameters
  - **max-age**: how long the rule is cached (seconds)
  - **includeSubDomains**: apply to all subdomains
  - **preload**: submit domain to browser preload list (must meet requirements)

# SSL-Stripping

- HTTPS Downgrade Attack
- Eve sits on public Wi-Fi, compromised router, malicious proxy
- Attack flow
  - Alice requests **http://example.com**
  - Server responds with 301/302 redirect to **https://example.com**
  - Eve intercepts and modifies the response
    - Removes the redirect
    - Rewrites links to stay **http://...**
    - Uses HTTPS to server but serves HTTP to victim
- Impact
  - Victim remains on **HTTP** while page appears “normal”
  - Credentials/actions can be **observed and modified** by attacker
- Mitigation
  - **HSTS** (and HSTS preload) forces HTTPS before any request is sent

## Mutual TLS

- Standard TLS
  - Client authenticates the **server** via server certificate (prevents MITM)
  - Client typically authenticates separately (password, cookie, token)
- Mutual TLS (mTLS)
  - Both sides authenticate:
    - Server presents a **server certificate**
    - Client presents a **client certificate** (X.509) during the handshake
  - Server verifies client certificate against a trusted CA and policy
  - Result: strong **client identity at the transport layer**
- Where it's used
  - Service-to-service in microservices / service mesh
  - Enterprise VPN-less "zero trust" access / internal APIs
  - High-trust partner integrations (B2B)

## Overview

- Security Concepts
- Transport Level Security
- **JSON Web Token**
- OAuth 2.0
- OpenID

## Overview

- JSON Web Token (JWT)
  - *Open standard (RFC 7519)*
  - *Mechanism to securely transmit information between parties as a JSON object.*
  - *Can be **verified** and **trusted** as it is **digitally signed**.*
- Basic concepts
  - *Compact*
    - *has a small size*
    - *can be transmitted via a URL, POST, HTTP header.*
  - *Self-contained*
    - *payload contains all required user information.*

## Use of JWT

- Authentication
  - *After user logs in, following requests contain JWT token.*
  - *Single Sign On widely uses JWT nowadays*
- Information Exchange
  - *Signature ensures senders are who they say they are.*
  - *Message integrity – signature calculated using the header and the payload.*

# JWT Structure

`<header>.<payload>.<signature>`

- Header

- Contains two parts, the type of the token (JWT) and the hashing algorithm being used (e.g. HMAC, SHA256, RSA).

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

- Payload

- Contains the claims, i.e. statements about an entity (e.g. user).
- Can be registered, public and private
- Registered and public should be defined in [IANA JSON Web Token Registry](#)

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "admin": true  
}
```

# JWT Structure (Cont.)

- Signature

- Signed encoded header, encoded payload and a secret.
- For example, signature using HMAC SHA256 algorithm

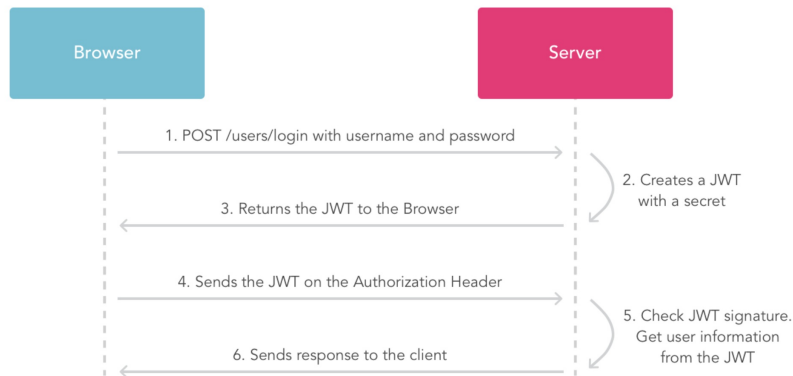
```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload),  
  secret)
```

- Example

- JWT is a three Base64-URL strings separated by dots

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4  
gRG9lIiwiaXNTb2NpYWwiOnRydWV9.  
4pcPyMD09o1PSyXnrXCjTwXyr4BsezdI1AVTmud2fU4
```

# How to use JWT



1. User sends username and password
2. Server verifies user, creates JWT token with a secret and a expiration time
3. Server sends JWT token back to the Browser
4. Browser sends JWT token on subsequent interactions

## Notes

- Authorization header does not behave the same as cookies!
- JWT should not contain secrets (passwords) as it can be read (on the client or if non-https connection is used)

# Standard Claims and Server-side Validation

- Common registered claims (RFC 7519)
  - **iss** (issuer): who issued the token
  - **aud** (audience): who the token is intended for (which API)
  - **sub** (subject): the user / entity identifier
  - **exp** (expiration): token must be rejected after this time
  - **nbf** (not before): token not valid before this time
  - **iat** (issued at): when the token was minted (useful for debugging / policies)
  - **jti** (JWT ID): unique identifier (useful for revocation lists)
- Server-side validation (do this on every request)
  - Verify the **signature** with the correct key
    - do not trust the token header blindly
  - Check **exp** (and allow small clock skew)
  - Check **iss** matches expected issuer
  - Check **aud** matches your service/API
  - If used, enforce **nbf/iat**; optionally check **jti** against a denylist



## Expiration and revocation

- **Expiration**
  - Tokens should be valid for a limited time
  - Use **exp** claim
    - Timestamp of the token expiration
    - Token should be checked on every request
- **Revocation**
  - Tokens (access tokens) usually stored in memory
  - Tokens should be refreshed using refresh tokens
    - Refresh tokens are stored in the DB
    - When you need to revoke access tokens stored them in the DB
    - You can expire all tokens by changing the secret on the server

## JWT Security Notes

- **JWT is usually signed, not encrypted**
  - Base64URL-encoded payload is readable by anyone who has the token
  - Do not put secrets (passwords/API keys) in JWTs
- **Algorithm handling**
  - Do not accept **alg: none**
  - Do not “trust the header” to choose any algorithm
    - allowlist expected algorithms per issuer
- **Browser storage tradeoffs**
  - Tokens in JS-accessible storage (e.g., localStorage)
    - vulnerable to XSS theft
  - Cookies reduce token theft but require CSRF defenses
    - SameSite/CSRF tokens
- **Size and performance**
  - Large JWTs increase request size and can hit proxy/header limits
    - keep claims minimal

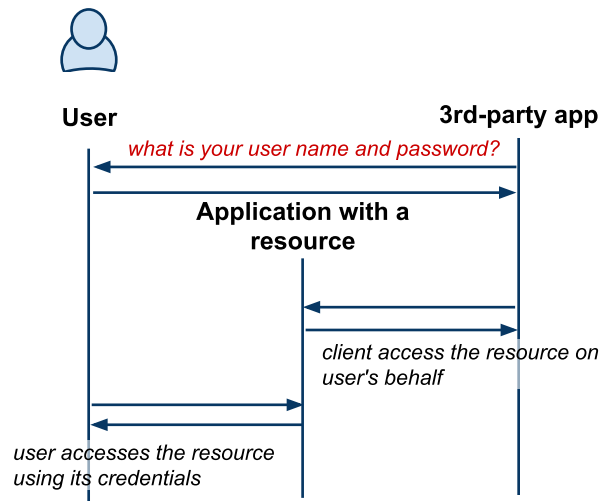
## Overview

- Security Concepts
- Transport Level Security
- JSON Web Token
- **OAuth 2.0**
  - *Client-side Web Apps*
  - *Server-side Web Apps*
- OpenID

## Motivation

- Cloud Computing – Software as a Service
  - *Users utilize apps in clouds*
    - *they access **resources** via Web browsers*
    - *they store their data in the cloud*
    - *Google Docs, Contacts, etc.*
  - *The trend is that SaaS are open*
    - *can be extended by 3rd-party developers through APIs*
    - *attract more users ⇒ increases value of apps*
  - *Apps extensions need to have an access to users' data*
- Need for a new mechanism to access resources
  - *Users can grant access to third-party apps without exposing their users' credentials*

## When there is no OAuth



- Users must share their credentials with the 3rd-party app
- Users cannot control what and how long the app can access resources
- Users must trust the app
  - In case of misuse, users can only change their passwords

## OAuth 2.0 Protocol

- OAuth Objectives
  - users can grant access to third-party applications
  - users can revoke access any time
  - supports:
    - client-side web apps (Authorization Code + PKCE),
    - server-side apps (Authorization Code), and
    - native (desktop) apps (Authorization Code + PKCE)
- History
  - Initiated by Google, Twitter, Yahoo!
  - Different, non-standard protocols first: ClientLogin, AuthSub
  - OAuth 1.0 – first standard, security problems, quite complex
  - OAuth 2.0 – new version, not backward compatible with 1.0
- Specifications and adoption
  - OAuth 2.0 Protocol [🔗](#)
  - OAuth 2.0 Google Support [🔗](#)

## Terminology

- **Client**
  - *a third-party app accessing resources owned by **resource owner***
- **Resource Owner** (also user)
  - *a person that owns a resource stored in the **resource server***
- **Authorization and Token Endpoints**
  - *endpoints provided by an **authorization server** through which a **resource owner** authorizes requests.*
- **Resource Server**
  - *an app that stores resources owned by a **resource owner***
  - *For example, contacts in Google Contacts*
- **Authorization Code**
  - *a code that a **client** uses to request **access tokens** to access resources*
- **Access Token**
  - *a code that a **client** uses to access resources*

## Overview

- Security Concepts
- Transport Level Security
- JSON Web Token
- OAuth 2.0
  - *Client-side Web Apps*
  - *Server-side Web Apps*
- OpenID

## Client-side Web Apps (SPA)

- Typical scenario
  - JavaScript/AJAX app running in a browser (public client)
    - cannot keep a client secret
    - Uses redirect-based login/consent via the user-agent
- Architecture
  - User-agent runs the HTML/JS client
  - Use **Authorization Code** flow with **PKCE** (recommended for SPAs)
    - prevents intercepted **code** from being exchanged by an attacker
- Basic Steps (high level)
  1. Client redirects user-agent to the authorization endpoint
    - **response\_type=code, code\_challenge, state**)
  2. Resource owner authenticates + grants access (or rejects)
  3. Authorization server redirects back with an **authorization code**
  4. Client exchanges **code** + **code\_verifier** for an **access\_token** (token endpoint)
  5. Client calls APIs using **Authorization: Bearer <access\_token>**
  6. When the access token expires, client obtains a new one (refresh token or re-auth, depending on design)

## PKCE: Proof Key for Code Exchange

- Goal
  - Prevents an attacker from exchanging a stolen authorization code
  - Important for public clients (SPAs, mobile/desktop apps)
    - They cannot keep **client\_secret**
- Key idea
  - Client creates a one-time secret **code\_verifier**
  - Client sends only a derived value **code\_challenge**
  - Client later proves possession by sending **code\_verifier**

## Step 1: Redirect to Authorization Endpoint

- Goal
  - Send the user-agent to the **authorization server** to start authentication + consent
  - Client includes parameters that bind the response to this browser session (**state**) and enable PKCE
- Key parameters
  - **response\_type=code** – request an authorization code (not an access token)
  - **client\_id** – identifies the SPA (public client)
  - **redirect\_uri** – where the auth server redirects back after login/consent
  - **scope** – requested permissions (least privilege)
  - **state** – CSRF protection + correlates request/response
  - **code\_challenge** + **code\_challenge\_method=S256** – PKCE (binds future token exchange)
    - **code\_verifier** = `BASE64URL_ENCODE(secure_random_bytes(32))`
    - **code\_challenge** = `BASE64URL_ENCODE( SHA256( ASCII(code_verifier) ) )`

- Example request

```
1 GET https://auth.example.com/oauth2/authorize?
2 response_type=code&
3 client_id=spa-client-123&
4 redirect_uri=https%3A%2F%2Fapp.mydomain.com%2Fcallback&
5 scope=openid%20profile%20email%20contacts_read&state=af0ifjsldkj
```

## Step 3: Redirect Back with Authorization Code

- What happens
  - After the user authenticates and approves requested scopes, the authorization server redirects the browser to the registered **redirect\_uri**
  - The redirect includes an authorization code (short-lived, one-time use) and the original **state**
- Example success callback

```
1 HTTP/1.1 302 Found
2 Location: https://app.mydomain.com/callback?
3 code=Sp1x10BeZQQYbYS6WxSbIA&
4 state=af0ifjsldkj
```
- Client (SPA) responsibilities
  - Verify **state** matches what was stored before the redirect (CSRF protection)
  - Extract **code** and proceed to Step 4 (token endpoint exchange using **code\_verifier**)
  - Authorization code is not an access token
- Error callback (user rejects or request invalid)

```
1 Location: https://app.mydomain.com/callback?
2 error=access_denied&
3 state=af0ifjsldkj
```

## Step 4: Exchange Code for Tokens

- Goal
  - Client exchanges the **authorization code** for an **access\_token** at the token endpoint
  - **PKCE** binds the exchange to the original browser session using **code\_verifier**
- Request
  - **POST** to the token endpoint
  - No **client\_secret** for a SPA

```
1 POST /oauth2/token HTTP/1.1
2 Host: auth.example.com
3 Content-Type: application/x-www-form-urlencoded
4 grant_type=authorization_code&
5 client_id=spa-client-123&
6 redirect_uri=https%3A%2F%2Fapp.mydomain.com%2Fcallback&
7 code=Sp1x10BeZQQYbYS6WxSbIA&
8 code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```
- Response (example)

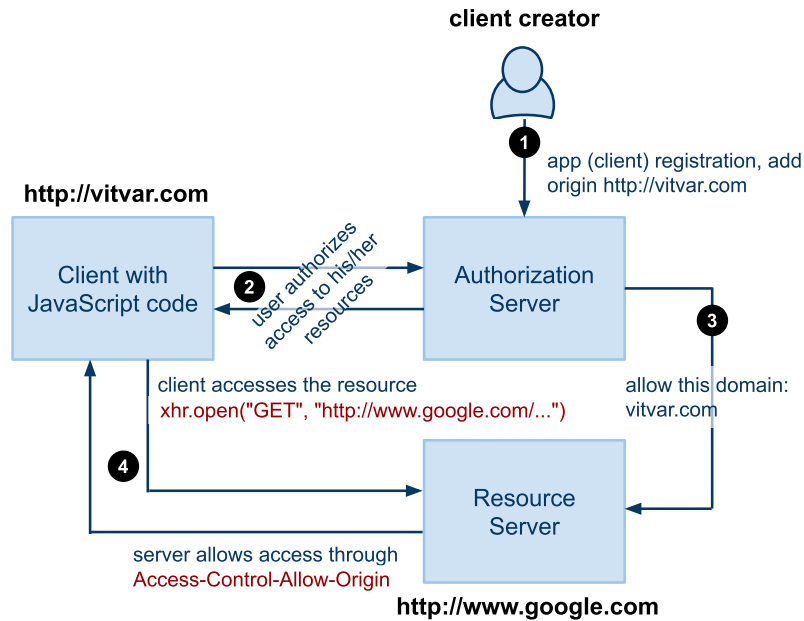
```
1 { "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
2   "token_type": "Bearer", "expires_in": 3600,
3   "scope": "openid profile email contacts.read" }
```
- Security notes
  - **code** is short-lived and single-use
  - If an attacker steals the **code** but not the **code\_verifier**, the exchange fails
  - Tokens should be sent only over **HTTPS**; never put tokens in **URL** queries

## Step 5: Call APIs with Access Token

- Goal
  - Use the **access\_token** to access protected resources on the **resource server**
  - Send tokens in the **Authorization** header (avoid URL query parameters)
- HTTP request (recommended)

```
1 GET /v1/contacts HTTP/1.1
2 Host: api.example.com Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
3 Accept: application/json
```
- What the resource server does
  - Validates the token (signature, **exp**, **iss**, **aud**, **scopes**)
  - Enforces permissions based on **scope** / **claims**
- Common responses
  - Success: **200 OK**
  - Missing/invalid token: **401 Unauthorized**
  - Valid token but insufficient scope/permission: **403 Forbidden**

# Cross-Origin Resource Sharing



— see *Same Origin and Cross-Origin* for details

# Example Application Registration

The screenshot shows the Google APIs console interface for the application "vitvar.com". The left sidebar contains navigation links: Overview, Services, Team, API Access (selected), Billing, Reports, and Quotas.

**API Access**

To prevent abuse, Google places limits on API requests. Using a valid OAuth token or API key allows you to exceed anonymous limits by connecting requests back to your project.

**Authorized API Access**

OAuth allows users to share specific data with you (for example, contact lists) while keeping their usernames, passwords, and other information private. [Learn more](#)

**Branding information**

The following information is shown to users whenever you request access to their private data.

Product name: w20-test  
Google account: t.vitvar@gmail.com

[Edit branding information...](#)

**Client ID for web applications**

|                     |                                                             |                                        |
|---------------------|-------------------------------------------------------------|----------------------------------------|
| Client ID:          | 621535099260.apps.googleusercontent.com                     | <a href="#">Edit settings...</a>       |
| Client secret:      | RxWM917Sv-7cyfWMW7KhNV9R                                    | <a href="#">Reset client secret...</a> |
| Redirect URIs:      | <code>http://vitvar.com/examples/oauth/callback.html</code> |                                        |
| JavaScript origins: | <code>http://example.org</code>                             |                                        |

[Create another client ID...](#)



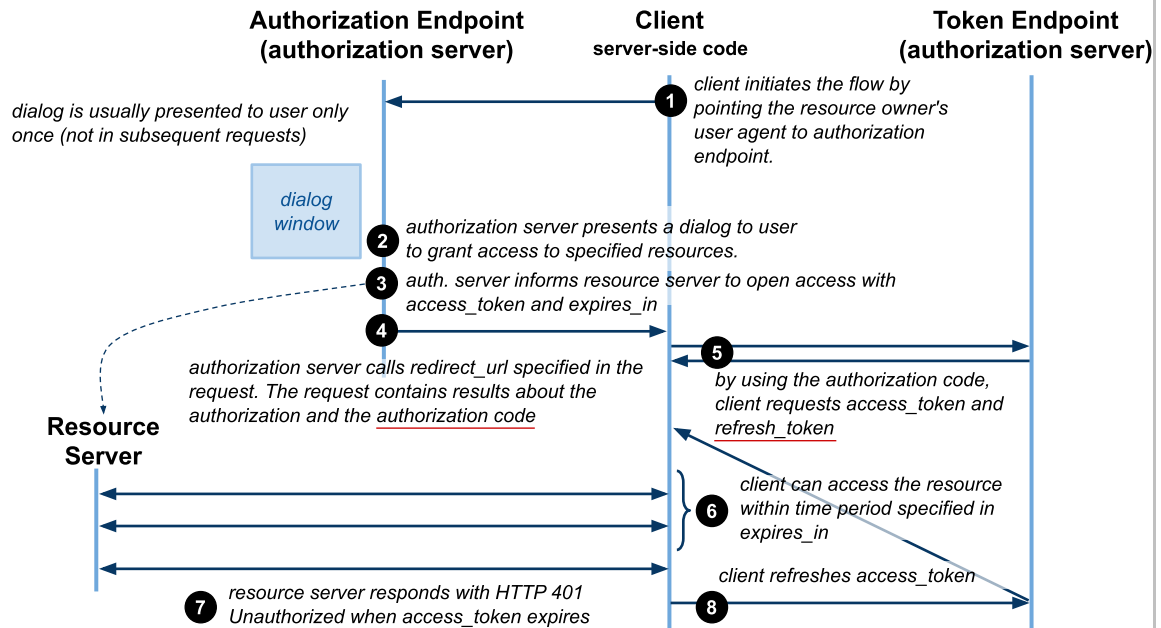
## Overview

- Security Concepts
- Transport Level Security
- JSON Web Token
- OAuth 2.0
  - *Client-side Web Apps*
  - *Server-side Web Apps*
- OpenID

## Server-side Web Apps

- Additional interactions
  - *server-side code (any language), the app can maintain the state*
  - *additional interactions, authorization code*
- Architecture
  - *Client at a server requests, remembers and refresh access tokens*
- Basic steps
  - *Client redirects user agent to the authorization endpoint*
  - *Resource owner grants access to the client or rejects the request*
  - *Authorization server provides **authorization code** to the client*
  - *Client requests **access and refresh tokens** from the auth. server*
  - *Client access the resource with the access token*
  - *When the token expires, client refreshes a token with refresh token*
- Advantages
  - *Access tokens not visible to clients, they are stored at the server*
  - *more secure, clients need to authenticate before they can get tokens*

# Server-side Web Apps Protocol



## Redirection – Step 1

- **Methods and Parameters**
  - same as for client-side app, except `response_type` must be `code`
- **Example**

```
1 | https://accounts.google.com/o/oauth2/auth?
2 | client_id=621535099260.apps.googleusercontent.com&
3 | redirect_uri=https://w20.vitvar.com/examples/oauth/callback.html&
4 | scope=https://www.google.com/m8/feeds&
5 | state=af0ifjsldkj&
6 | response_type=code
```

## Callback + Access Token Request – steps 4, 5

- Callback
  - authorization server calls back **redirect\_uri**
  - client gets the **code** and requests **access\_token**
  - client verifies the **state** parameter
  - example (resource owner grants access):  
**`https://w20.vitvar.com/examples/oauth/callback.html?code=4/P7...`**
  - when user rejects → same as client-side access
- Access token request
  - **POST** request to token endpoint  
→ example Google token endpoint:  
**`https://accounts.google.com/o/oauth2/token`**
  - ```
1 POST /o/oauth2/token HTTP/1.1
2 Host: accounts.google.com
3 Content-Type: application/x-www-form-urlencoded
4
5 code=4/P7q7W91a-oMsCeLvIaQm6bTrgtp6&
6 client_id=621535099260.apps.googleusercontent.com&
7 client_secret=XTHhXh1S2UggvyWGwDk1EjXB&
8 redirect_uri=https://w20.vitvar.com/examples/oauth/callback.html&
9 grant_type=authorization_code
```

## Access Token (cont.)

- Access token response
  - Token endpoint responds with **access\_token** and **refresh\_token**
  - ```
1 { "access_token" : "1/ffAGRNJru1FTz70BzhT3Zg",
2   "expires_in"   : 3920,
3   "refresh_token" : "1/6BMfW9j53gdGimsixUH6kU5RsR4zwI9lUVX-tqf8JXQ" }
```
- Refreshing a token
  - **POST** request to the token endpoint with **grant\_type=refresh\_token** and the previously obtained value of **refresh\_token**
  - ```
1 POST /o/oauth2/token HTTP/1.1
2 Host: accounts.google.com
3 Content-Type: application/x-www-form-urlencoded
4
5 client_id=21302922996.apps.googleusercontent.com&
6 client_secret=XTHhXh1S1UNGvyWGwDk1EjXB&
7 refresh_token=1/6BMfW9j53gdGimsixUH6kU5RsR4zwI9lUVX-tqf8JXQ&
8 grant_type=refresh_token
```
- Accessing a resource is the same as in the client-side app

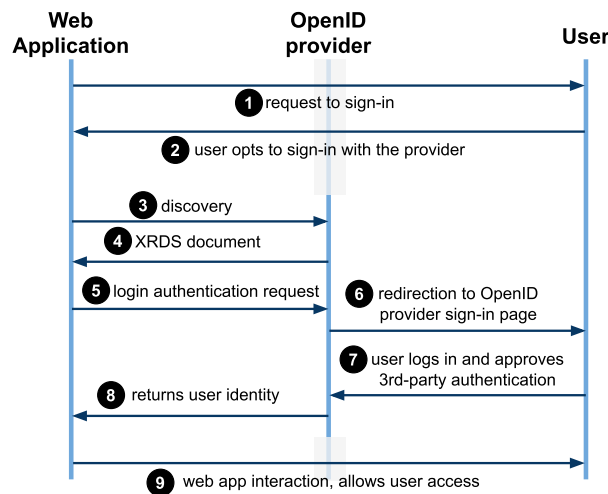
## Overview

- Security Concepts
- Transport Level Security
- JSON Web Token
- OAuth 2.0
- **OpenID**
  - *OpenID Connect*

## OpenID Protocol

- Motivation – many user accounts
  - *users need to maintain many accounts to access various services*
  - *multiple passwords problem*
- Objectives
  - *allows apps to utilize an OpenID provider*
    - *a third-party authentication service*
    - *federated login*
  - *users have one account with the OpenID provider and use it for apps that support the provider*
- OpenID providers
  - *it is a protocol, anybody can build a provider*
  - *Google, Yahoo!, Seznam.cz, etc.*
- Specification
  - *OpenID Protocol* [🔗](#)

## Interaction Sequence



- Discovery – discovery of a service associated with a resource
- XRDS – eXtensible Resource Descriptor Sequence
  - *format for discovery result*
  - *developed to serve resource discovery for OpenID*
  - *Web app retrieves endpoint to send login authentication requests*

## Login Authentication Request – Step 5

- Example Google OpenID provider

```
1 https://www.google.com/accounts/o8/id
2 ?openid.ns=http://specs.openid.net/auth/2.0
3 &openid.return_to=https://www.example.com/checkauth
4 &openid.realm=http://www.example.com/
5 &openid.assoc_handle=ABSmpf6DNMw
6 &openid.mode=checkid_setup
```

- Parameters

- **ns** – *protocol version (obtained from the XRDS)*
- **mode** – *type of message or additional semantics (**checkid\_setup** indicates that interaction between the provider and the user is allowed during authentication)*
- **return\_to** – *callback page the provider sends the result*
- **realm** – *domain the user will trust, consistent with **return\_to***
- **assoc\_handle** – *"log in" for web app with openid provider*

*\* Not all fields shown, check the OpenID spec for the full list of fields and their values*

## Login Authentication Response – Step 8

- User logs in successfully

```
1 http://www.example.com/checkauth
2 ?openid.ns=http://specs.openid.net/auth/2.0
3 &openid.mode=id_res
4 &openid.return_to=http://www.example.com:8080/checkauth
5 &openid.assoc_handle=ABSmpf6DNMw
6 &openid.identity=https://www.google.com/accounts/o8/id?id=ACyQatiscWwqs4UQV_L
```

- Web app will use **identity** to identify user in the application
- response is also signed using a list of fields in the response (not shown in the listing)

- User cancels

```
1 http://www.example.com/checkauth
2 ?openid.mode=cancel
3 &openid.ns=http://specs.openid.net/auth/2.0
```

*\* Not all fields shown, check the OpenID spec for the full list of fields and their values*

## Overview

- Security Concepts
- Transport Level Security
- JSON Web Token
- OAuth 2.0
- OpenID
  - *OpenID Connect*

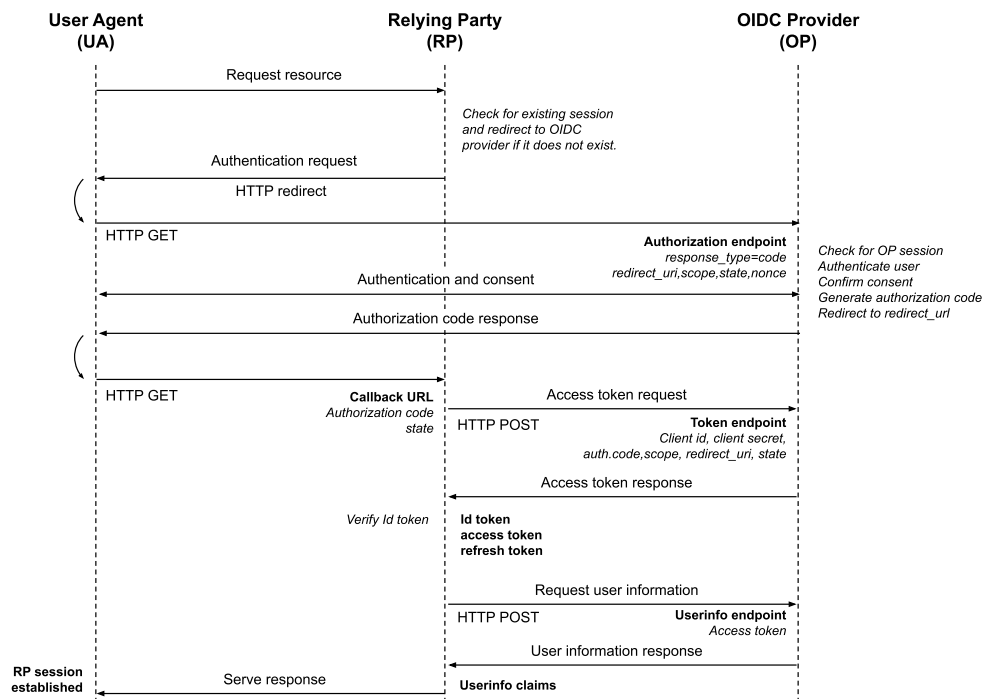
## OpenID Connect (OIDC)

- Simple identity layer on top of the OAuth 2.0 protocol
  - *Authorization Server to verify identity of users*
  - *Clients can obtain basic profile information about users*
  - *OAuth answers: “what can this app access?”*
  - *OIDC answers: “who is the user?”*
- OIDC vs OpenID
  - *OIDC does many of the same tasks as OpenID 2.0*
  - *API-friendly*
    - *can be used by native and mobile applications*
  - *Robust signing and encryption mechanisms*
  - *Native integration with OAuth 2.0.*
- Defined by OpenID open standard
  - *OpenID Connect*

## OIDC Roles and Tokens

- Roles (same actors as OAuth, plus OIDC specifics)
  - *End-User: the user who is logging in*
  - *Relying Party (RP): the client application (OAuth client)*
  - *OpenID Provider (OP): the identity provider*
    - *OAuth authorization server*
- Tokens
  - *ID Token (new in OIDC): proves authentication; contains identity claims*
    - *almost always a JWT signed by the OP*
  - *Access Token (OAuth): used to call APIs / resource server*
  - *Refresh Token (optional): used to get new tokens without re-auth*

# Interaction sequence



## OIDC Step 1: Authentication Request

- **Same redirect as OAuth**, but request includes OIDC parameters
  - **scope** must include **openid** (otherwise it is just OAuth)
  - **nonce** binds the ID Token to this browser session (replay protection)
  - **state** protects against CSRF and correlates request/response
  - For SPAs/native apps
    - use Authorization Code + PKCE
- Example (Auth Code + PKCE)

```

1 GET https://idp.example.com/authorize?
2   response_type=code&
3   client_id=rp-client-123&
4   redirect_uri=https%3A%2F%2Fapp.mydomain.com%2Fcallback&
5   scope=openid%20profile%20email&
6   state=af0ifjsldkj&
7   nonce=n-0S6_WzA2Mj&
8   code_challenge=E9Melhoa20wvFrEMTJguCHaoeK1t8URWbuGJSstw-cM&
9   code_challenge_method=S256
    
```



## OIDC Step 2: User Authentication + Consent

- What happens at the OpenID Provider (OP)
  - User-agent displays the OP login page (if not already signed in)
  - End-User authenticates
    - password, MFA, passkey, etc. (handled by the OP)
  - OP asks for consent (if needed)
    - based on requested **scope** (e.g., **openid**, **profile**, **email**)
- Important notes
  - Client never sees the user's password
  - OP establishes its own session with the browser (often via cookies)
  - Result:** OP is ready to issue an authorization response (next step)

## OIDC Step 3: Redirect Back with Authorization Code

- After successful authentication (and consent), OP redirects back
  - Redirect target is the registered **redirect\_uri**
  - Response includes an **code** and the original **state**
- Example success callback

```
1 | HTTP/1.1 302 Found
2 | Location: https://app.mydomain.com/callback?
3 | code=Sp1x10BeZQQYbYS6WxSbIA&
4 | state=af0ifjsldkj
```
- Client responsibilities
  - **Verify state** matches what was stored in Step 1 (CSRF protection)
  - Extract **code** and continue to Step 4 (token endpoint exchange)
- Error callback (user rejects or request invalid)

```
1 | Location: https://app.mydomain.com/callback?
2 | error=access_denied&
3 | state=af0ifjsldkj
```

## OIDC Step 4: Token Request

- Client exchanges the authorization **code** at the token endpoint

- Example (server-side)

```
1 POST /oauth2/token HTTP/1.1
2 Host: idp.example.com
3 Content-Type: application/x-www-form-urlencoded
4 grant_type=authorization_code&
5 client_id=server-client-123&
6 client_secret=CLIENT_SECRET&
7 redirect_uri=https%3A%2F%2Fserver.mydomain.com%2Fcallback&
8 code=Sp1X10BeZQYbYS6WxSbIA
```

- Token response

- **access\_token** (*OAuth*)
- **id\_token** (*OIDC – identity token*)
- **refresh\_token** (*optional*)

- Example response

```
{ "access_token": "S1AV32hkKG",
  "token_type": "Bearer",
  "expires_in": 3600,
  "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

## ID Token (JWT) – Typical Claims

- Registered claims to validate
  - **iss** – *expected issuer (the OP)*
  - **aud** – *must contain your **client\_id** (intended audience)*
  - **exp** – *not expired (allow small clock skew)*
  - **iat** – *issued at (optional policy)*
  - **nonce** – *must match the nonce sent in Step 1 (replay protection)*
- User identity
  - **sub** – *stable identifier for the End-User at this OP*
  - *Profile claims are often requested via scopes*
    - **profile, email, phone, address**

## Request User Information (optional)

- Purpose
  - Retrieve additional user profile claims from the OP's **UserInfo** endpoint
  - Use when you need more claims than what is in the **id\_token** (or need fresher data)
- Important notes
  - Call **/userinfo** using the **access token** (not the **id\_token**)
  - Returned claims depend on granted scopes (e.g., **profile**, **email**)
- Example request

```
1 GET /userinfo HTTP/1.1
2 Host: idp.example.com
3 Authorization: Bearer access_token
4 Accept: application/json
```
- Example response

```
{ "sub": "248289761001",
  "name": "Jane Doe",
  "given_name": "Jane",
  "family_name": "Doe",
  "email": "jane.doe@example.com" }
```